

DevOps Interview

Question & Answer Guide

Sanket Ajay Chopade

B.Tech 2025 · DevOps Engineer Intern, Hisan Labs

AWS · Kubernetes · Terraform · Docker · Jenkins · GitHub Actions
Prometheus · Grafana · Argo CD · Amazon EKS · Fargate

■ How to use this guide:

Every answer is tailored to your background. Read the answer, then practice saying it aloud.
For situational questions, adapt the scenario to your actual internship experience.

■ 1. Amazon Web Services (AWS)

Q: Which AWS services have you used most frequently?

Answer:

During my internship at Hisan Labs, I worked with the following AWS services regularly:

- Amazon EKS – Managed Kubernetes clusters for container orchestration.
- EC2 – Virtual machines used in some environments.
- IAM – Identity and Access Management for controlling permissions.
- VPC – Virtual Private Cloud for network isolation.
- S3 – Object storage for artifacts, Terraform state, and application assets.
- CloudWatch – Monitoring logs, metrics, and alarms.
- ECR – Elastic Container Registry for storing Docker images.
- AWS Load Balancer Controller – For managing ALB/NLB ingress traffic.
- Amazon Bedrock & SageMaker – Basic exposure for GenAI services.

Q: Explain your experience with Amazon EKS.

Answer:

I provisioned and managed Amazon EKS clusters as part of both my internship and personal projects.

Key activities included:

- Creating EKS clusters using Terraform (eksctl or AWS provider).
- Configuring node groups and Fargate profiles for workload scheduling.
- Setting up kubeconfig and kubectl to interact with the cluster.
- Deploying applications via Argo CD in a GitOps workflow.
- Configuring AWS Load Balancer Controller to expose services externally.
- Implementing IRSA (IAM Roles for Service Accounts) for pod-level permissions.

■ *Tip: Mention a specific cluster size or workload type if asked for more detail.*

Q: What is the difference between EC2 and EKS?

Answer:

- EC2 is a virtual machine (IaaS). You manage the OS, runtime, scaling, and everything else manually.
- EKS is a managed Kubernetes service. AWS manages the control plane (API server, etcd). You manage worker nodes and workloads.
- EKS abstracts away cluster bootstrapping. With EC2 alone, you would need to set up Kubernetes yourself.
- EKS is best for containerised, microservice workloads. EC2 is flexible for any type of workload.
- EKS with Fargate removes node management entirely — you only pay per pod.

Q: How do Security Groups work?

Answer:

Security Groups act as a virtual firewall for AWS resources such as EC2, RDS, and EKS nodes.

Key characteristics:

- They are stateful — if inbound traffic is allowed, return traffic is automatically allowed.
- Rules are ALLOW only — there is no explicit DENY.
- Applied at the instance/ENI level, not the subnet level (NACLs operate at subnet level).
- You can reference other security groups as the source/destination instead of CIDR ranges.
- Best practice: follow the principle of least privilege — open only required ports.

Q: What is IAM and why is it important?

Answer:

IAM stands for Identity and Access Management. It controls who (authentication) can do what (authorization) in AWS.

Key components:

- Users – Individual accounts with long-term credentials.
- Groups – Collections of users sharing the same permissions.
- Roles – Temporary credentials assumed by services, EC2, Lambda, or EKS pods (via IRSA).
- Policies – JSON documents that define allowed or denied actions on resources.

IAM is critical for security. In EKS I used IRSA to give pods only the specific S3 or DynamoDB permissions they needed, following least-privilege.

Q: Explain VPC architecture.

Answer:

A VPC (Virtual Private Cloud) is an isolated network inside AWS.

Core components:

- Subnets – Subdivisions of the VPC CIDR block. Public subnets have a route to the Internet Gateway; private subnets do not.
- Internet Gateway (IGW) – Allows public internet access for resources in public subnets.
- NAT Gateway – Allows private subnet resources to initiate outbound internet connections without being directly reachable.
- Route Tables – Define how traffic is directed between subnets and gateways.
- Security Groups & NACLs – Control traffic at the instance and subnet level respectively.

In EKS deployments, worker nodes typically reside in private subnets. The Load Balancer sits in public subnets, and NAT Gateway allows nodes to pull container images.

Q: What is CloudWatch used for?

Answer:

- Metrics – Collecting performance data from AWS services (CPU, memory via agent, EKS metrics).
- Logs – Centralised log collection from EC2, Lambda, EKS control plane, and application logs.
- Alarms – Triggers notifications or Auto Scaling actions when metrics breach thresholds.
- Dashboards – Visual representation of metrics across services.
- Events/EventBridge – Respond to state changes in AWS resources.

During my internship, CloudWatch was used alongside Datadog. CloudWatch handled AWS-native metrics and EKS control plane logs, while Datadog provided a unified view.

■ ■ 2. Terraform & Infrastructure as Code

Q: What is Infrastructure as Code (IaC)?

Answer:

IaC is the practice of managing and provisioning infrastructure through machine-readable configuration files instead of manual processes.

Benefits:

- Consistency – Same code produces the same infrastructure every time.
- Version control – Infrastructure changes are tracked in Git like application code.
- Automation – No manual clicks in the AWS console; CI/CD can apply changes.
- Reusability – Modules allow reuse across teams and environments.
- Disaster recovery – Infrastructure can be recreated quickly from code.

Q: Why Terraform instead of CloudFormation?

Answer:

- Multi-cloud – Terraform supports AWS, Azure, GCP, and hundreds of providers. CloudFormation is AWS-only.
- HCL syntax – More readable than YAML/JSON CloudFormation templates.
- State management – Terraform maintains a state file enabling drift detection.
- Modules – Rich module registry (registry.terraform.io) for reusable components.
- Community – Larger community and more third-party providers available.

In a multi-cloud or hybrid environment, Terraform is the natural choice. For AWS-only shops, CloudFormation integrates natively with services like Service Catalog.

Q: Explain the Terraform workflow.

Answer:

- terraform init – Downloads providers and modules; initialises the backend.
- terraform plan – Creates an execution plan showing what will be created, changed, or destroyed.
- terraform apply – Executes the plan and provisions/modifies real infrastructure.
- terraform destroy – Removes all resources managed by the configuration.

State is updated after every apply. I always ran plan first in CI and only applied after reviewing the diff.

Q: What is Terraform state and why is it important?

Answer:

Terraform state is a JSON file that tracks the mapping between configuration and real-world resources.

Importance:

- Enables Terraform to calculate the difference (diff) between desired and current state.
- Required for destroy operations — Terraform knows what to remove.
- Stores resource IDs returned by AWS (e.g., VPC ID, instance ID).
- Without state, every apply would attempt to create resources from scratch.

Q: What is remote state and how do you handle state locking?

Answer:

Remote state stores the terraform.tfstate file in a shared backend instead of locally.

I used S3 as the backend with DynamoDB for state locking:

- S3 stores the state file and supports versioning for rollback.
 - DynamoDB table stores a lock entry — prevents two engineers from running apply simultaneously.
 - Configuration: terraform { backend "s3" { bucket, key, region, dynamodb_table } }
 - If a lock is stuck after a failed apply, terraform force-unlock is used.
-

Q: What are Terraform modules and how do you manage multiple environments?

Answer:

Modules are reusable packages of Terraform configuration, similar to functions in programming.

I created modules for VPC, EKS cluster, and IAM roles.

For multiple environments (dev/staging/prod):

- Directory structure: environments/dev, environments/staging, environments/prod.
 - Each environment has its own terraform.tfvars with different variable values.
 - Shared modules are referenced from a central modules/ directory.
 - Workspaces can also be used, but separate directories are cleaner for large teams.
-

■ 3. Docker & Containers

Q: Why use Docker? What is the difference between Docker and Virtual Machines?

Answer:

Docker packages applications and their dependencies into portable containers.

Key differences vs VMs:

- VMs include a full OS kernel — Docker containers share the host OS kernel, making them lightweight.
- Containers start in seconds; VMs take minutes.
- Docker images are typically megabytes; VM images are gigabytes.
- VMs provide stronger isolation (hypervisor boundary). Containers share the kernel.
- Docker is better for microservices and CI/CD; VMs are better for full OS-level isolation.

Q: Explain Docker architecture.

Answer:

- Docker Daemon (dockerd) – Background service managing images, containers, networks, volumes.
- Docker CLI – Command-line interface that sends commands to the daemon via REST API.
- Docker Image – Read-only template built from a Dockerfile. Layers are cached for efficiency.
- Container – A running instance of an image with a writable layer on top.
- Docker Registry – Storage for images (Docker Hub, Amazon ECR, GitHub Container Registry).
- Docker Network – bridge (default), host, none, overlay (Swarm) for container communication.

Q: What are multi-stage builds and why did you use Distroless images?

Answer:

Multi-stage builds allow using multiple FROM instructions in one Dockerfile.

Stage 1 (build stage): Install dependencies and compile code.

Stage 2 (final stage): Copy only the compiled binary — no build tools included.

Benefits of Distroless images:

- Contain only the runtime (e.g., Java or Node runtime) — no shell, package manager, or OS tools.
- Drastically reduces attack surface — no bash means no shell injection.
- Smaller image size — faster pulls and less storage.
- Better security compliance for production workloads.

■ *Tip: Example: FROM gcr.io/distroless/java17 as the final stage for a Java app.*

Q: How do you troubleshoot a container failure?

Answer:

- `docker ps -a` – List all containers including stopped ones.
- `docker logs` – View stdout/stderr logs.
- `docker inspect` – Detailed JSON with exit code, mounts, network settings.
- `docker exec -it /bin/sh` – Open a shell inside a running container.
- Check exit code: code 0 = success, 1 = general error, 137 = OOM killed, 143 = SIGTERM.
- Review Dockerfile: wrong CMD, missing ENV variables, incorrect port exposure.

■ 4. Kubernetes & Amazon EKS

4a. Kubernetes Fundamentals

Q: What is Kubernetes and why use it?

Answer:

Kubernetes (K8s) is an open-source container orchestration platform that automates deployment, scaling, and management of containerised applications.

Reasons to use Kubernetes:

- Self-healing – Automatically restarts failed containers and reschedules them on healthy nodes.
- Horizontal scaling – HPA scales pods up/down based on CPU/memory metrics.
- Service discovery & load balancing – Built-in DNS and kube-proxy.
- Rolling updates & rollbacks – Zero-downtime deployments.
- Declarative configuration – Desired state defined in YAML manifests.

Q: Explain Kubernetes architecture.

Answer:

Kubernetes has a Control Plane and Worker Nodes.

Control Plane components:

- kube-apiserver – Front end for the Kubernetes API; all CLI and tool communication goes here.
- etcd – Distributed key-value store holding the entire cluster state.
- kube-scheduler – Assigns pods to nodes based on resource requests, affinity rules, taints/tolerations.
- kube-controller-manager – Runs controllers (ReplicaSet, Node, Deployment, etc.).

Worker Node components:

- kubelet – Agent on each node; ensures containers in pods are running as specified.
- kube-proxy – Manages network rules (iptables/IPVS) for service routing.
- Container runtime – containerd or CRI-O runs the actual containers.

Q: What is a Pod, Deployment, and ReplicaSet?

Answer:

- Pod – Smallest deployable unit in Kubernetes. Wraps one or more containers sharing network and storage.
- ReplicaSet – Ensures a specified number of pod replicas are running at all times. Replaces failed pods.
- Deployment – Higher-level abstraction over ReplicaSet. Provides rolling updates, rollback history, and declarative updates.

Typical hierarchy: Deployment → ReplicaSet → Pod.

Q: What is a Service in Kubernetes?

Answer:

A Service provides a stable IP address and DNS name for a group of pods selected by a label selector.

Service types:

- ClusterIP (default) – Internal access only within the cluster.
- NodePort – Exposes the service on a static port on each node IP.
- LoadBalancer – Provisions a cloud load balancer (e.g., ALB on EKS).
- ExternalName – Maps a service to an external DNS name.

Q: What is Ingress and how is it different from a Service?

Answer:

Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services inside the cluster.

Differences from Service (LoadBalancer):

- A LoadBalancer Service creates one cloud LB per service. Ingress routes multiple services through a single LB.
 - Ingress supports path-based (/api → service-a, /web → service-b) and host-based routing.
 - Ingress requires an Ingress Controller (e.g., AWS Load Balancer Controller, NGINX).
 - Ingress is more cost-efficient on cloud — one ALB serves many services.
-

Q: What is HPA (Horizontal Pod Autoscaler)?

Answer:

HPA automatically scales the number of pod replicas based on observed metrics.

How it works:

- The HPA controller queries the Metrics Server (or custom metrics adapter) at regular intervals.
- It compares current metric value vs. target value and calculates the desired replica count.
- Formula: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetricValue} / \text{desiredMetricValue}))$.
- Supports CPU, memory (with metrics-server), and custom metrics (Prometheus via kube-adapter).

■ *Tip: Always set resource requests on pods — HPA cannot calculate ratios without them.*

4b. Kubernetes Troubleshooting

Q: A Pod is stuck in Pending state. What do you do?

Answer:

- `kubectl describe pod` — Look for Events at the bottom.
 - Common causes: Insufficient CPU/memory on nodes → scale up the node group.
 - Node selector or affinity rules not matched — check node labels.
 - Persistent Volume Claim (PVC) not bound — check PVC status and StorageClass.
 - Taints on nodes — check if the pod has matching tolerations.
 - On EKS Fargate: check the Fargate profile selector matches the pod's namespace/labels.
-

Q: Pod is in CrashLoopBackOff. What do you check?

Answer:

- `kubectl logs --previous` — View logs from the last crashed container.
 - `kubectl describe pod` — Check last exit code (137 = OOM, 1 = app error).
 - Verify environment variables and ConfigMaps are correctly mounted.
 - Check liveness probe configuration — overly aggressive probes restart healthy pods.
 - Ensure the container's CMD/ENTRYPOINT is correct.
 - Review resource limits — pod may be OOM-killed if memory limit is too low.
-

Q: Service is not accessible. How do you troubleshoot?

Answer:

- `kubectl get svc` — Verify service exists and has the correct ClusterIP/ExternalIP/NodePort.
 - `kubectl describe svc` — Check the Endpoints field; if empty, label selectors do not match any pods.
 - `kubectl get endpoints` — Confirm pods are listed as endpoints.
 - For LoadBalancer: check cloud LB is provisioned and DNS is resolving.
 - For Ingress: `kubectl describe ingress` — check if ALB/NGINX has assigned an address.
 - Check Security Groups and NACLs if on EKS — ports must be open.
-

Q: What happens when a worker node fails?

Answer:

- The node controller marks the node as NotReady after the node monitor grace period (default 40s).
 - After the pod eviction timeout (default 5 min), pods are evicted and rescheduled on healthy nodes.
 - PodDisruptionBudgets (PDB) limit how many pods can be unavailable simultaneously.
 - On EKS, the node group can auto-replace the failed node via Auto Scaling Group.
 - Stateful workloads with PVCs may not reschedule until the old pod is fully terminated.
-

■ 5. Jenkins & CI/CD Pipelines

Q: Explain CI/CD and what pipelines you created.

Answer:

CI (Continuous Integration): Automatically build and test code every time it is pushed to a repository.

CD (Continuous Delivery/Deployment): Automatically deliver tested code to staging or production.

Pipelines I built at Hisan Labs:

- Code commit → GitHub webhook triggers Jenkins pipeline.
- Stage 1: Checkout — pull latest code from GitHub.
- Stage 2: Build — compile / build Docker image.
- Stage 3: Test — unit tests and SonarQube static analysis.
- Stage 4: Push — push Docker image to ECR/DockerHub.
- Stage 5: Deploy — update Kubernetes manifests or trigger Argo CD sync.

Q: What are Jenkins agents and how do they work?

Answer:

Jenkins agents (formerly slaves) are nodes that execute pipeline jobs offloaded from the controller.

Types:

- Static agents – Permanent nodes configured in Jenkins with fixed SSH or JNLP connections.
- Dynamic agents – Kubernetes plugin spins up a pod for each build and terminates it after completion.
- Docker agents – agent { docker { image 'maven:3.8' } } runs the build inside a container.

Using Kubernetes-based agents keeps the controller light and scales build capacity automatically.

Q: How did you integrate SonarQube in Jenkins?

Answer:

- Installed SonarQube Scanner plugin in Jenkins.
- Configured SonarQube server URL and token under Jenkins → Configure System.
- Added a withSonarQubeEnv('sonar-server') block in the Jenkinsfile to run the scanner.
- Added a Quality Gate stage: waitForQualityGate() — pipeline fails if quality gate is not passed.
- SonarQube checked for code smells, bugs, vulnerabilities, and code coverage thresholds.

Q: What happens when a Jenkins build fails?

Answer:

- Pipeline is marked as FAILURE; downstream stages are skipped by default.
 - post { failure { ... } } block sends notification (email, Slack, Teams).
 - I investigated via Console Output in Jenkins UI or build logs.
 - Common causes: compilation error, failed tests, Docker build error, invalid credentials.
 - Fixed the code/config, pushed again, and the webhook automatically triggered a new build.
-

■ 6. GitHub Actions

Q: Explain GitHub Actions workflow files.

Answer:

Workflows are YAML files stored in `.github/workflows/`. They define automated processes.

Key components:

- on – Trigger: push, pull_request, schedule, workflow_dispatch.
- jobs – One or more jobs; each runs on a runner machine.
- steps – Sequential steps within a job; each step runs a command or uses an action.
- uses – References a reusable action (e.g., actions/checkout@v4).
- env / secrets – Environment variables and encrypted secrets (referenced as `${{ secrets.AWS_ACCESS_KEY_ID }}`).

Q: What are GitHub Actions runners?

Answer:

- Runners are machines (VMs or containers) that execute workflow jobs.
- GitHub-hosted runners: ubuntu-latest, windows-latest, macos-latest — managed by GitHub.
- Self-hosted runners: machines you register with GitHub; useful for VPC-level AWS access or custom tooling.
- In my GitOps project I used ubuntu-latest runners for building Docker images and pushing to ECR.

Q: Jenkins vs GitHub Actions — which would you choose and why?

Answer:

Both are valid. The choice depends on the use case:

- GitHub Actions is simpler for repos hosted on GitHub — no separate server to maintain.
- GitHub Actions has native integration with GitHub events (PR checks, branch protection).
- Jenkins is better for complex enterprise pipelines needing fine-grained control and many plugins.
- Jenkins requires infrastructure management; GitHub Actions is serverless.
- For new cloud-native projects on GitHub, I would choose GitHub Actions. For large existing Jenkins infrastructure, I would maintain Jenkins.

■ 7. Monitoring & Observability

Q: What is the difference between monitoring and observability?

Answer:

- Monitoring: Watching predefined metrics and alerts to know WHEN something is wrong.
 - Observability: The ability to understand WHY something is wrong by examining logs, metrics, and traces.
 - Observability = Metrics + Logs + Traces (the three pillars).
 - Monitoring tells you 'the CPU is high'. Observability helps you trace it to a specific request or code path.
-

Q: How does Prometheus collect metrics? What are exporters?

Answer:

Prometheus uses a pull-based model — it scrapes HTTP endpoints (usually /metrics) at regular intervals.

Exporters:

- Exporters are agents that translate system metrics into Prometheus format.
 - node_exporter – Exposes Linux system metrics (CPU, memory, disk, network).
 - kube-state-metrics – Exposes Kubernetes object state (pod count, deployment replicas).
 - cAdvisor – Container-level CPU and memory metrics.
 - Custom exporters can be written for application-specific metrics.
-

Q: What Grafana dashboards did you create? What alerts would you configure?

Answer:

Dashboards I created in Grafana:

- Kubernetes cluster overview: node CPU/memory, pod count, deployment health.
- Application dashboard: request rate, error rate, p95/p99 latency (RED method).
- Infrastructure dashboard: disk I/O, network throughput, JVM heap (for Java services).

Alerts I configured:

- Pod restarts > 5 in 10 minutes → PagerDuty/Slack alert.
 - Node memory usage > 85% for 5 minutes → scale warning.
 - HTTP error rate > 1% → immediate on-call alert.
 - Deployment stuck (pod not ready > 10 min) → deployment failure alert.
-

Q: Why use Datadog if Prometheus/Grafana already exists?

Answer:

- Datadog is a full SaaS observability platform — no infrastructure to manage.
 - It unifies metrics, logs, traces, and APM in a single tool with powerful correlation.
 - Datadog provides out-of-the-box integrations with 500+ services and richer alerting.
 - At Hisan Labs, Datadog was used for application-level APM and distributed tracing, while Prometheus handled Kubernetes infrastructure metrics.
 - Datadog's Log Management with ML-based anomaly detection was valuable for production debugging.
-

■ 8. AWS GenAI Services (Bedrock, Q, SageMaker)

Q: What is Amazon Bedrock?

Answer:

Amazon Bedrock is a fully managed service that provides access to Foundation Models (FMs) from leading AI companies via a single API.

Key features:

- Supports models from Anthropic (Claude), Meta (Llama), Mistral, Amazon (Titan), Cohere, and others.
- No ML infrastructure to manage — invoke models via API calls.
- Supports fine-tuning and Retrieval-Augmented Generation (RAG) with Knowledge Bases.
- Agents in Bedrock can take multi-step actions using tools.

Q: What is Amazon Q Developer?

Answer:

- Amazon Q Developer is an AI-powered coding assistant integrated into IDEs (VS Code, JetBrains, Eclipse).
- It provides code suggestions, auto-completion, and security vulnerability scanning.
- For DevOps, it can generate Terraform code, CloudFormation templates, and AWS CLI commands.
- Amazon Q also integrates with the AWS Console to answer questions about your AWS environment.

Q: Difference between Amazon Bedrock and SageMaker?

Answer:

- Bedrock: Use pre-built Foundation Models via API — no ML expertise needed. Best for building GenAI applications quickly.
 - SageMaker: Full ML platform for training, fine-tuning, and deploying custom ML models. Requires ML/data science expertise.
 - Bedrock = consume AI. SageMaker = build and train AI.
 - Use Bedrock for chatbots, text generation, RAG. Use SageMaker for custom model training on proprietary data.
-

■ 9. GitOps CI/CD Pipeline Project

Q: Explain the GitOps project architecture.

Answer:

This project implemented a fully automated GitOps-based CI/CD pipeline.

Architecture:

- Application code in GitHub repository A. Kubernetes manifests in a separate repository B.
- Developer pushes code → GitHub Actions workflow triggers automatically.
- GitHub Actions: checkout code → run tests → build Docker image → push to Amazon ECR.
- GitHub Actions then updates the image tag in the Kubernetes manifest repo (repo B) via a commit.
- Argo CD monitors repo B. When it detects a change, it syncs the new manifests to EKS.
- EKS cluster rolls out the new deployment with the updated image.

■ *Tip: Separating app code and manifests is a GitOps best practice — the manifest repo is the single source of truth.*

Q: What is the source of truth in GitOps?

Answer:

In GitOps, the Git repository containing Kubernetes manifests is the single source of truth for the desired cluster state.

Key principle:

- No manual `kubectl` apply in production — all changes go through Git (Pull Request → review → merge).
 - Argo CD continuously compares the live cluster state against the Git manifest state.
 - If someone manually changes a resource in the cluster, Argo CD marks it as `OutOfSync` and can auto-revert it.
 - This ensures auditability, traceability, and disaster recovery (just replay Git history).
-

Q: How do rollbacks work in this GitOps setup?

Answer:

- Since all manifests are in Git, rollback = reverting a Git commit.
 - `git revert` in the manifest repo creates a new commit restoring the previous image tag.
 - Argo CD detects the revert commit and syncs the cluster back to the previous version.
 - Argo CD also supports one-click rollback to a previous sync in its UI (History and Rollback menu).
 - This is faster and safer than manually running `kubectl rollout undo`.
-

Q: Why Argo CD over Flux or manual `kubectl`?

Answer:

- Argo CD provides a visual UI showing sync status, resource health, and deployment history.
 - It supports multi-cluster deployments from a single control plane.
 - App of Apps pattern allows managing multiple applications declaratively.
 - Argo CD integrates with RBAC and SSO for secure team access.
 - Manual `kubectl` is error-prone and leaves no audit trail. Argo CD enforces the GitOps workflow.
-

■ 10. Serverless Game Hosting Platform (EKS + Fargate)

Q: Why use AWS Fargate for EKS?

Answer:

- Fargate is serverless Kubernetes — no EC2 worker nodes to provision, patch, or manage.
 - AWS manages node-level infrastructure; you only define CPU and memory per pod.
 - Automatically scales per-pod — no pre-provisioning of node capacity.
 - Cost-efficient for bursty workloads: pay only when pods are running.
 - Improved security: each pod runs in its own isolated compute environment (microVM).
-

Q: What is IRSA and why use OIDC?

Answer:

IRSA = IAM Roles for Service Accounts. It allows Kubernetes pods to assume AWS IAM roles.

How it works:

- EKS cluster exposes an OIDC (OpenID Connect) issuer URL.
- An IAM role is created with a trust policy that references the OIDC provider and a specific service account.
- The service account is annotated with the IAM role ARN.
- The pod running with that service account receives temporary AWS credentials automatically via the OIDC token.

Why OIDC: It replaces the insecure practice of storing AWS access keys in pod environment variables. Credentials are short-lived and scoped to the specific service account.

Q: What are the limitations of Fargate on EKS?

Answer:

- DaemonSets are not supported — sidecar containers must be used instead.
 - Privileged containers are not allowed.
 - No GPU support on Fargate.
 - Node-level SSH/exec is not possible — you can only exec into the pod.
 - Stateful workloads with large persistent volumes may have limitations.
 - Cold start latency — Fargate pods take slightly longer to start than EC2-based pods.
-

■ 11. E-Commerce Internship (Hisan Labs)

Q: What was your day-to-day routine as a DevOps Intern?

Answer:

- Attended daily standup (Agile ceremony) to report task progress and blockers.
 - Monitored CI/CD pipeline runs; investigated and fixed failed builds.
 - Assisted with managing Kubernetes deployments and checking pod health.
 - Worked on Terraform scripts for provisioning or updating cloud infrastructure.
 - Reviewed monitoring dashboards (Datadog) and investigated alerts.
 - Collaborated with developers to resolve environment and deployment issues.
 - Updated runbooks and internal documentation for processes I performed.
-

Q: How was code moved from development to production?

Answer:

The team followed a structured pipeline approach:

- Developers raised Pull Requests targeting the main/development branch on GitHub.
 - GitHub webhook triggered a Jenkins pipeline for automated build and test.
 - SonarQube quality gate was checked — failing quality gate blocked the merge.
 - After merge, a deployment pipeline pushed the image to ECR and updated the staging environment.
 - After QA sign-off, a release pipeline promoted the change to production.
 - Kubernetes rolling updates ensured zero-downtime deployments.
-

Q: How did you collaborate with developers?

Answer:

- Used Jira/sprint board for task tracking — DevOps tasks were part of the same sprint.
 - Communicated via Slack channels for real-time collaboration.
 - Developers opened tickets for environment setup, pipeline issues, or new service onboarding.
 - Participated in sprint retrospectives to identify process improvements.
 - Helped developers debug container issues and environment variable mismatches.
-

■ 12. Situational / Scenario-Based Questions

Q: Website is down after a deployment. What is your approach?

Answer:

I follow a structured incident response:

- 1. Check monitoring dashboards (Datadog/Grafana) for alerts and error rate spikes.
 - 2. `kubectl get pods -n` — check for `CrashLoopBackOff` or `Pending` pods.
 - 3. `kubectl describe pod` / `kubectl logs` — find the root cause.
 - 4. Check recent deployments: `kubectl rollout history deployment/`.
 - 5. If deployment is the cause: `kubectl rollout undo deployment/` to rollback immediately.
 - 6. Check Argo CD for sync errors if using GitOps.
 - 7. Communicate status to stakeholders during the incident.
 - 8. Post-incident: write a blameless post-mortem and fix the root cause.
-

Q: Terraform apply fails. What steps would you take?

Answer:

- Read the error message carefully — Terraform is usually descriptive.
 - `terraform plan` again after any fixes to verify the plan is correct.
 - Common causes: AWS permission denied (check IAM policy), resource already exists (import it), dependency issues (check `depends_on`).
 - If state is locked, check for a concurrent apply; use `terraform force-unlock` only if certain no other apply is running.
 - Check AWS Service Quotas if you are hitting limits (e.g., max VPCs, max EIPs).
-

Q: Terraform state file is corrupted. What would you do?

Answer:

- Do NOT run `terraform apply` immediately — this could destroy or duplicate resources.
 - If using S3 backend with versioning: restore the previous version of the state file from S3.
 - Run `terraform refresh` to reconcile state with real infrastructure.
 - As a last resort, use `terraform import` to re-import existing resources.
 - Set up S3 versioning and DynamoDB locking to prevent this in future.
-

Q: CPU usage suddenly spikes on a Kubernetes cluster. What do you investigate?

Answer:

- `kubectl top pods --all-namespaces` — identify the pod consuming the most CPU.
 - `kubectl top nodes` — check if the issue is node-level or pod-level.
 - Check Grafana/Prometheus dashboards for the trend (was it gradual or sudden?).
 - `kubectl logs` — look for runaway processes or infinite loops.
 - Check if a new deployment went out (correlate with deployment history).
 - If a specific pod is misbehaving, restart it: `kubectl rollout restart deployment/`.
 - Review HPA settings — if replicas are not scaling, CPU limits may be too low.
-

Q: Argo CD sync fails. What do you check?

Answer:

- Open Argo CD UI → Application → check the sync error message.
 - Common causes: invalid YAML in the manifest (missing field, wrong indentation).
 - RBAC issue: Argo CD service account lacks permission to create/update a resource.
 - Resource already exists with a different owner — use `kubectl` to inspect the existing resource.
 - Webhook from GitHub not reaching Argo CD — check network/firewall rules.
 - `kubectl logs -n argocd deployment/argocd-application-controller` for deeper errors.
-

Q: Users report slowness but all pods are healthy. What do you investigate?

Answer:

- Check application-level metrics: response time (p95/p99 latency), request queue depth.
 - Check database connection pool saturation — slow DB queries are a common cause.
 - Network latency — check if inter-AZ traffic or external API calls are slow.
 - Check JVM GC pauses or memory pressure if running Java applications.
 - Review recent code deployments — a new feature may have introduced a slow query or inefficient loop.
 - Use distributed tracing (Datadog APM / Jaeger) to identify the slow span in the request path.
-

■ 13. Behavioral Questions

Q: Tell me about a technology you learned quickly.

Answer:

I learned Argo CD and GitOps principles rapidly when assigned to the GitOps pipeline project.

Approach:

- Spent the first two days going through the official Argo CD documentation and hands-on labs.
- Set up a local kind cluster to experiment before touching the real environment.
- Built a proof-of-concept GitOps pipeline within a week.
- Key insight: GitOps is not just a tool — it is a philosophy of using Git as the source of truth.

Result: Successfully implemented the complete pipeline in three weeks, which is now used in production deployments.

Q: Tell me about a challenge during your internship.

Answer:

One challenge was debugging a Docker image that worked locally but failed in EKS.

Investigation:

- Local: used amd64 (x86) architecture. EKS nodes: also amd64 — so architecture was not the issue.
 - Found the issue: Distroless image lacked a required shared library that the application expected.
 - Fix: Added the library in the multi-stage build's final stage.
 - Lesson: Always test Docker images in an environment matching production. Use `docker inspect` and `docker run` locally before pushing.
-

Q: Describe a mistake you made and how you fixed it.

Answer:

During a Terraform apply on the staging environment, I accidentally ran it on the wrong workspace and modified a production Security Group.

Actions taken:

- Immediately identified the change using `terraform state list` and AWS Console.
- Reverted the Security Group rule manually via AWS CLI.
- Ran `terraform apply` again on the correct workspace to reconcile state.
- Root cause: missing workspace validation in the pipeline.
- Fix: Added an explicit check (`terraform workspace show; exit 1 if not staging/prod as expected`) as a pipeline guard.

Lesson: Always confirm the workspace/environment before running `terraform apply`, especially in automated pipelines.

Q: How do you learn new technologies?

Answer:

- Official documentation first — it is the most accurate and up-to-date source.
 - Hands-on practice: spin up a lab environment (local kind/minikube, or a free-tier AWS account).
 - Build a small project that solves a real problem rather than following tutorials passively.
 - Read GitHub issues and community forums to understand real-world pain points.
 - Follow DevOps communities: CNCF Slack, r/devops, KubeWeekly newsletter.
-

■ 14. Fresher & HR Questions

Q: Why DevOps instead of Development?

Answer:

I am genuinely passionate about the intersection of software delivery and infrastructure.

Key reasons:

- I enjoy both writing code (scripts, pipelines) and understanding the systems that run it.
- DevOps solves the friction between development speed and operational stability — a problem that directly impacts business.
- The breadth of technologies — cloud, containers, networking, security — keeps me continuously learning.
- During my B.Tech projects I realised I was more excited setting up the CI/CD pipeline than writing the application itself.

Q: Why should we hire you? What differentiates you?

Answer:

- Practical, hands-on experience from day one — I have built real GitOps pipelines and managed EKS clusters, not just theoretical knowledge.
- I understand the end-to-end software delivery lifecycle from code commit to production monitoring.
- My internship at Hisan Labs exposed me to production environments with real stakes, Agile teams, and enterprise tooling.
- I am a fast learner who builds side projects to apply new skills immediately.
- I bring energy, curiosity, and a strong problem-solving mindset.

Q: What is your greatest strength and weakness?

Answer:

Strength:

- My greatest strength is breaking down complex problems systematically and finding automation opportunities.
- I am persistent — I do not give up when troubleshooting. I follow a structured approach until I find the root cause.

Weakness:

- I sometimes spend too long perfecting a solution when a good-enough solution would suffice.
- I am working on this by setting time-boxes for tasks and consulting seniors earlier when stuck.

Q: Where do you see yourself in 5 years?

Answer:

- In 2 years: Proficient DevOps / Cloud Engineer contributing independently to complex infrastructure projects.
- In 5 years: Senior DevOps Engineer or Platform Engineer, leading the design of cloud-native platforms, mentoring juniors, and possibly working towards AWS Solutions Architect or CKA certification.
- Long-term: Contribute to building platforms that enable development teams to ship faster and more reliably.

Q: Do you have any questions for us?

Answer:

- What does a typical day look like for a DevOps Engineer on your team?
- What is your current cloud and infrastructure stack? Are there any major migrations or modernisation projects planned?
- How does your team handle on-call rotations and incident management?
- What are the opportunities for learning and certification support?
- How do development and DevOps teams collaborate here?

■ *Tip: Always have 2–3 genuine questions ready. It signals enthusiasm and preparation.*

■ Quick Preparation Roadmap

Priority	Topic	Focus Areas	Suggested Time
■■■	Kubernetes	Architecture, Troubleshooting, RBAC, HPA, Ingress	30%
■■■	AWS	EKS, IAM/IRSA, VPC, CloudWatch, ECS vs EKS	25%
■■	Terraform	Workflow, State, Modules, Remote Backend	15%
■■	Jenkins & CI/CD	Pipeline stages, Agents, SonarQube, GitHub webhooks	10%
■■	Docker	Multi-stage builds, Distless, Troubleshooting	10%
■	Linux	chmod, systemctl, journalctl, ps, netstat, grep	5%
■	Monitoring	Prometheus pull model, Grafana, Datadog APM	5%

■ Practice answering every question aloud (or record yourself). Confidence comes from rehearsal, not just reading. Best of luck, Sanket! ■